MicroGoals – Secure Goal Management Backend API

Team ID : SWTID-2026-5881

Team Size : 4

Team Leader : [PARVEEN A & 24csparveen066@gmail.com]

Team member :

[POORNIMA L & 24cspoornima68@gmail.com]

[RITHIKA V & 24csrithika.v79@gmail.com]

[NIVEDITHA R & 24csniveditha065@gmail.com]

1.Project Overview:

MicroGoals is a secure backend RESTful API developed using Node.js, Express.js, and MongoDB. It enables users to register, log in, and manage their personal goals through CRUD operations. The system uses JWT authentication and middleware to ensure data security and user-specific access control.

The system includes:

- User Registration & Login
- JWT-based Authentication
- CRUD Operations for Goals
- Protected Routes
- Centralized Error Handling

The main objective is to provide a secure and scalable backend system for digital goal management.

2.Project Instruction:

- The project follows REST API principles.
- Authentication is handled using JWT.
- MongoDB is used as the database.
- APIs are tested using Thunder Client.

- The project follows MVC architecture.

3.Pre-requisites:

Before running the project, the following tools are required:

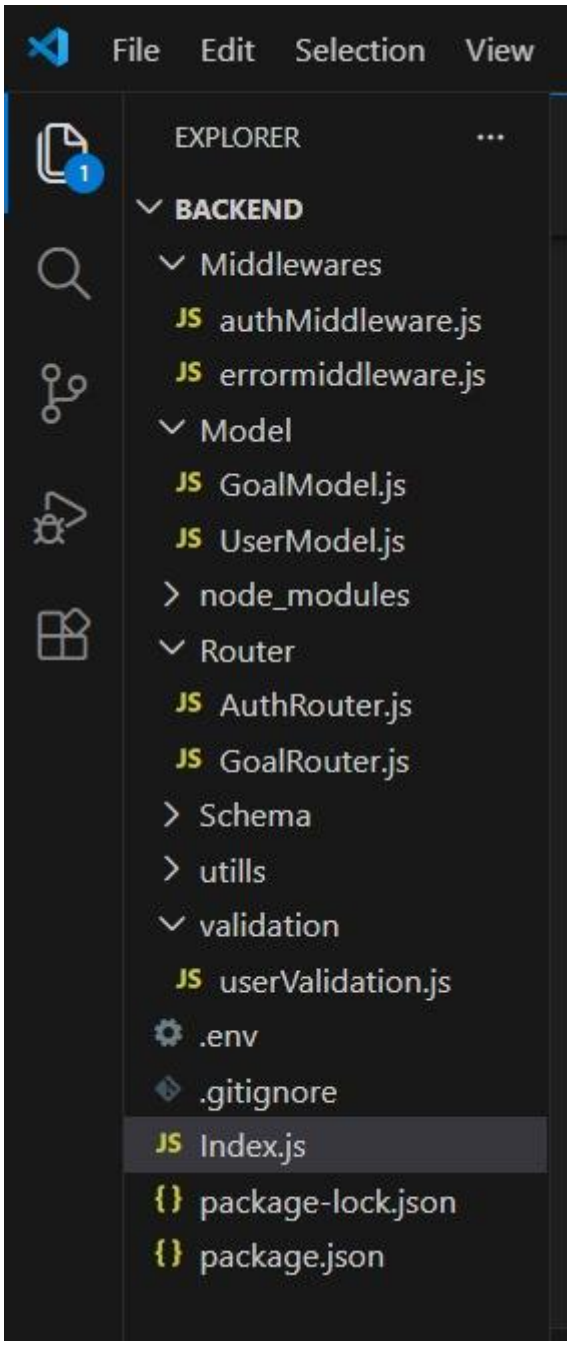
- Node.js (v18 or higher)
- Express.js
- MongoDB (Local / Atlas)
- VS Code
- Thunder Client (Extension)
- Basic knowledge of JavaScript & REST APIs

4.Project setup and configuration:

🔷 Folder Structure

```
MicroGoals
 ├ middleware
 ├ authMiddleware.js
 ├  errorMiddleware.js
 ├Schema/
 ├User.js
 ├GoalSchema.js
 ├models/
 ├userModel.js
 ├goalModel.js
 ├routes
 ├authRoutes.js
 ├ goalRoutes.js
 ├ .env
 ├ Index.js
```

├package.json



◆ Project Setup:

1. npm init -y

2. Install dependencies:

npm install express mongoose jsonwebtoken bcryptjs dotenv

3. Start server:

node server.js

5.Backend development:

● Created Express Server An Express server was set up to handle API requests, connect routes, and enable JSON parsing for backend operations

```
Go  Run  Terminal  Help                                  Backend
JS Index.js ●   .env
JS Index.js > ...
1   require("dotenv").config();
2
3   const express = require("express");
4   const mongoose = require("mongoose");
5   const cors = require("cors");
6
7   const app = express();
8
9   // middleware
10  app.use(cors());
11  app.use(express.json());
12  app.use(express.urlencoded({ extended: true }));
13
14  // error middleware
15  const errormiddleware = require("./Middlewares/errormiddleware");
16
17  // routes
18  const authroute = require("./Router/AuthRouter");
19  const GoalRouter = require("./Router/GoalRouter");
20
21  // environment variables
22  const PORT = process.env.PORT || 8000;
23  const mongo_url = process.env.MONGO_URL;
```

```
Go  Run  Terminal  Help                                  Backend
JS Index.js ●   .env
                                  string | undefined
JS Index.js > ...
22  const PORT = process.env.PORT || 8000;
23  const mongo_url = process.env.MONGO_URL;
24
25  // MongoDB connection
26  console.log(mongo_url);
27
28  mongoose.connect(mongo_url)
29  .then(() => console.log("MongoDB Connected"))
30  .catch((err) => console.log(err));
31
32  // routes
33  app.use("/", authroute);
34  app.use("/", GoalRouter);
35
36  // error middleware
37  app.use(errormiddleware);
38
39  // start server
40  app.listen(PORT, () => {
41    console.log(`Server running on port ${PORT}`);
42  });
```

● Connected MongoDB

For Connection with MongoDB:-

```
//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

- Implemented Authentication APIs

User registration and login APIs were created with password hashing and JWT token generation for secure authentication.

```
JS Index.js    JS AuthRouter.js ×
const express = require("express");
const router = express.Router();
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const userValidation = require("../validation/userValidation");
const userModel = require("../Model/UserModel");
const genrateToken = require("../utills/genratetoken");
router.post("/signup", async (req, res, next) => {
  try {
    const { error } = userValidation.validate(req.body);
    if (error) {
      return res.status(400).json({
        success: false,
        message: "enter the data correctly",
      });
    }
    const { name, email, password } = req.body;
    const exist = await userModel.findOne({ email });
    if (exist) {
      return res.status(400).json({ message: "User already exists" });
    }
    const hashpassword = await bcrypt.hash(password, 10);
    const user = new userModel({
      name: name,
      email: email,
      password: hashpassword,
    });
    await user.save();
    const token = genrateToken(user._id);
    res.status(200).json({
      success: true,
      message: "user is created",
      token: token,
      data: user,
    });
  } catch (err) {
    return next(err);
  }
});

router.post("/login", async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = await userModel.findOne({ email });

    if (!user) {
      const err = new Error("user not found");
      err.status = 400;
      return next(err);
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      const err = new Error("Wrong Password");
      err.status = 400;
      return next(err);
    }
    const token = genrateToken(user._id);
    res.status(200).json({
      success: true,
      message: "login succesfully",
      token: token,
      data: user,
    });
  } catch (error) {
    return next(error);
  }
```

- Implemented Goal CRUD APIs

Goal CRUD APIs were implemented to allow authenticated users to perform full Create, Read, Update, and Delete operations on their personal goals. These APIs follow RESTful architecture principles and are designed to ensure secure and structured data management.

The Create API enables users to add new goals to the database. Each goal is associated with a specific user using a userId reference, ensuring user-specific data isolation.

The Read API allows authenticated users to retrieve only their own goals from the database. Data filtering is applied using the authenticated user's ID to prevent unauthorized data access.

The Update API enables users to modify existing goals. Proper validation checks ensure that only the owner of the goal can update it.

The Delete API allows users to permanently remove their goals from the system. Secure route protection ensures that users cannot delete goals belonging to others.

All CRUD routes are protected using Authentication Middleware, which verifies the JWT token before granting access. If the token is missing or invalid, the system returns a 401 Unauthorized error.

Additionally, Error Middleware is implemented to handle runtime and validation errors centrally. It captures exceptions from CRUD operations and returns consistent JSON responses with appropriate HTTP status codes such as 200, 201, 400, 401, and 404.

This structured implementation ensures secure data handling, proper authorization, and reliable API behavior across all goal management operations.

6.Project execution:

Project Execution Steps

1. Start MongoDB

2. Run server

3. Test APIs in Thunder Client

4. Register User

5. Login and get JWT

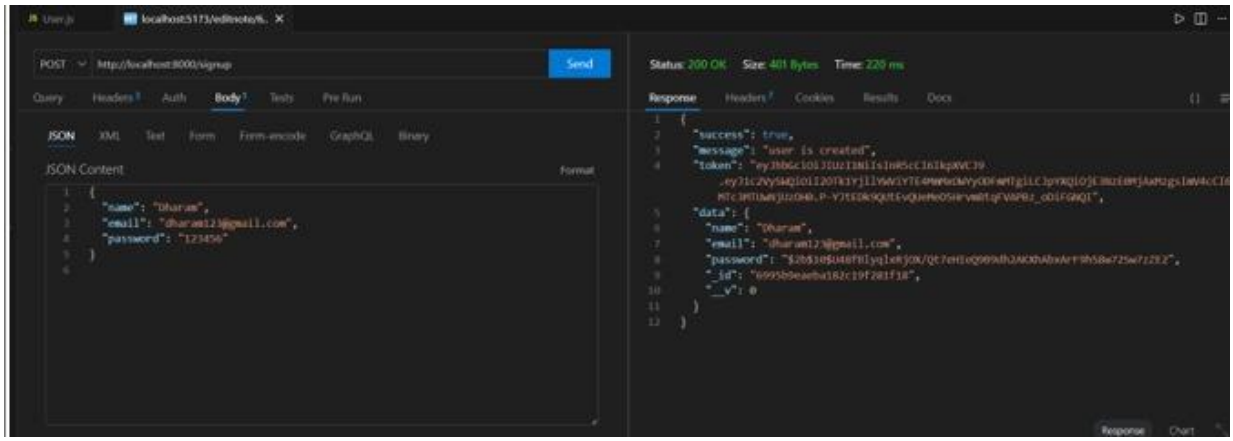
6. Perform CRUD operations

7.Project Output Screenshots:

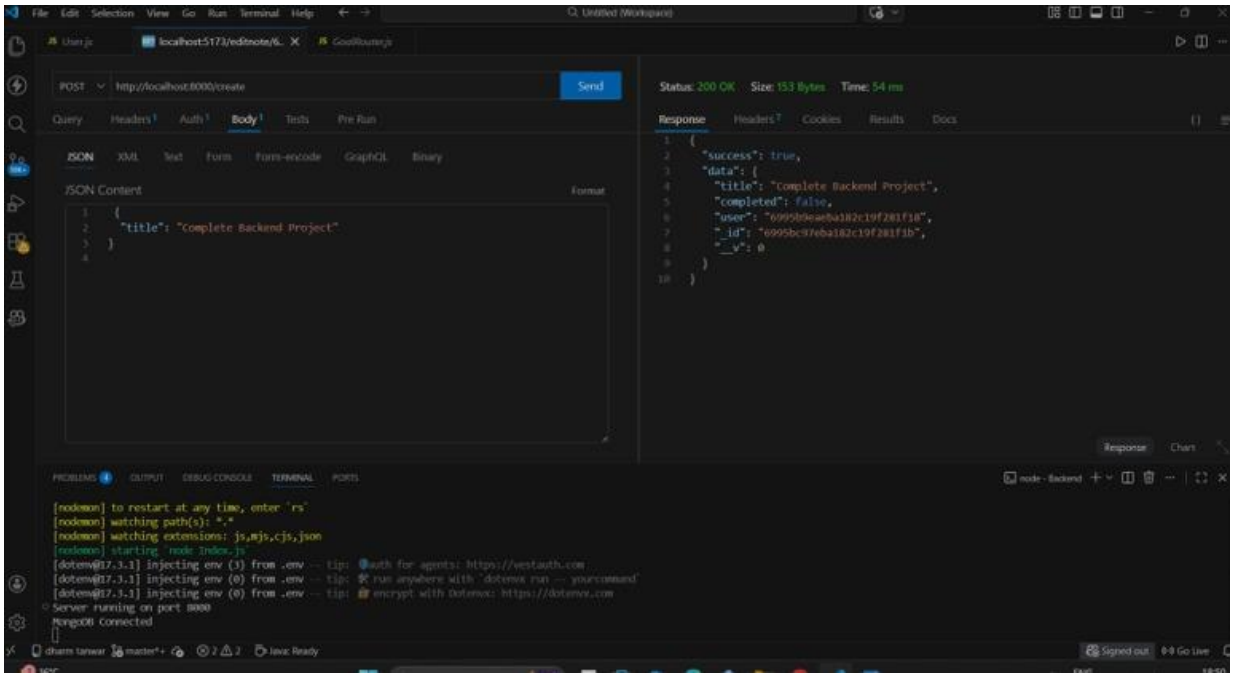
● Successful User Registration(Signup/Login)

● In this Api call the user register or login and token was generated

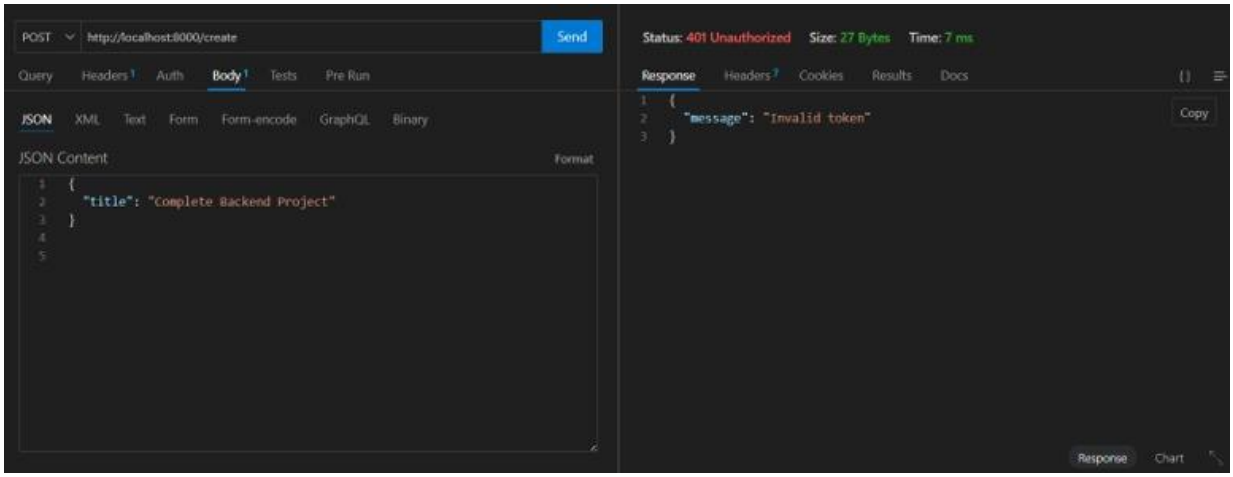
● JWT Token Response After successful login user the api generate the token and these token was sent with every request to every api



- Goal Created Successfully
- After the login and token is generated the user give post request to api for creating there goals and goal was save in MongoDB which was the database



- Protected Route Working
- If the user does not have token which was generated by backend api then he cannot access the api so Route are protectedZ



8.Project Demo Video:

https://drive.google.com/file/d/1z50Ld-YMGRf8nHbUPVwHbIUis_2cUYdy/view?usp=drive_link